SimpleStorage -Smart Contract

Overview

What is a Smart Contract?

A smart contract is a self-executing computer program stored on a blockchain.

- It contains rules written in code (usually Solidity on Ethereum).
- Once deployed, it runs automatically when conditions are met, without the need for intermediaries.
- They are immutable (cannot be changed after deployment) and transparent (anyone can view the code and transactions).

Example: In this project, the smart contract simply stores and retrieves a message.

Key Features of Smart Contracts

1. Automation – Executes predefined rules automatically when triggered.
2. Trustless – No third party needed; the blockchain enforces execution.
3. Transparency – Code and transactions are visible on the blockchain explorer.
4. Immutability – Once deployed, the contract logic cannot be altered.
5. Security – Cryptographic mechanisms secure contract execution.
6. Global Accessibility – Can be interacted with from anywhere in the world using a wallet.
7. Cost Efficiency – Reduces intermediaries, though transaction (gas) fees apply.

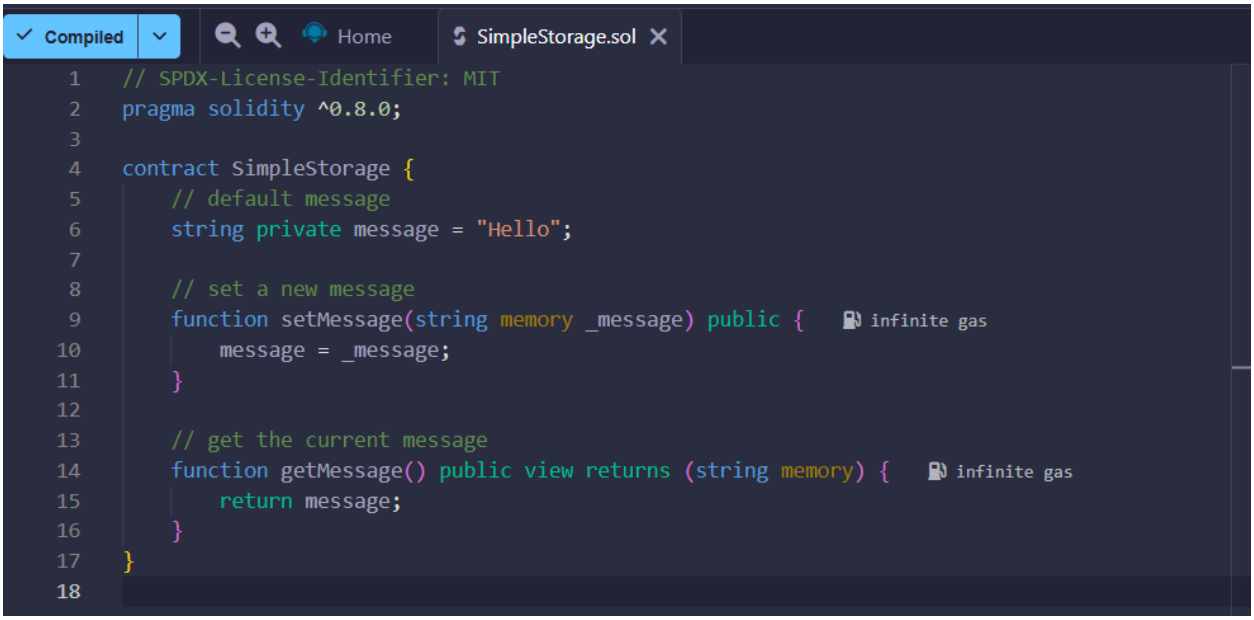
This document demonstrates a **basic getter–setter smart contract** written in Solidity.
The contract allows a user to **store a message** (setMessage) and **retrieve it** (getMessage).

As part of this project, I have researched about **five EVM-compatible test networks**:

- Ethereum (Sepolia)
- Polygon (Mumbai/Amoy)

- Avalanche (Fuji)
- Binance Smart Chain (BSC Testnet)
- Gnosis (Chiado)

Smart Contract Code



Explanation of Code

1.// SPDX-License-Identifier: MIT

- This is a license identifier.
- Required by Solidity compilers for legal clarity.
- MIT means the code is open-source and can be reused with attribution.

2.pragma solidity ^0.8.0;

- Tells the compiler which version of Solidity to use.
- ^0.8.0 means any version 0.8.x or higher up to (but not including) 0.9.0.
- Ensures compatibility and prevents accidental breaking changes.

3.contract SimpleStorage { ... }

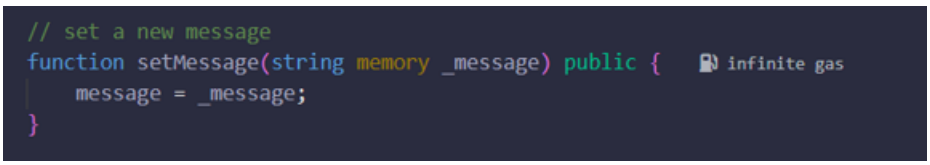
- Defines a contract named SimpleStorage.
- A contract in Solidity is like a class in traditional programming — it contains state (data) and functions.

4.string private message = "Hello";

- Declares a state variable:
  - string → data type (text).
  - private → visibility (only functions inside this contract can directly access it).
  - message → variable name.
- Initialized with default value "Hello".
- Stored on-chain permanently (until changed).

5.function setMessage(string memory _message) public { ... }

- A function that sets/updates the stored message.
- Parameters:
  - string memory _message → input value stored temporarily in memory during execution.
- public → anyone can call this function.
- **Inside the function:**



Updates the message state variable with the new value.

- **Since this modifies blockchain state, it is a transaction → costs gas.**

function getMessage() public view returns (string memory) { ... }

- A function that retrieves the current message.
- public → anyone can call it.
- view → means it does not modify blockchain state, it only reads.
- returns (string memory) → specifies it outputs a string.
- Inside the function:
- **return message;**

```
// get the current message
function getMessage() public view returns (string memory) {    infinite gas
    return message;
}
```

  - **Returns the current value stored in message.**
- **Calling this is free because it's just a read-only call.**

Summary of Function Behavior

- setMessage() → write to blockchain (transaction, costs gas).
- getMessage() → read from blockchain (no gas if called locally).

Other Languages for Smart Contracts (besides Solidity)

1. Vyper (Ethereum)

- Python-like language for the Ethereum Virtual Machine (EVM).
- Syntax is simpler and more restrictive than Solidity.
- Designed for security and auditability (fewer features, less chance of bugs).

2. Rust (Solana, NEAR, Polkadot, Aptos, etc.)

- Used in non-EVM chains like Solana, NEAR Protocol, and Polkadot/Substrate.

- Very powerful and memory-safe.
- Example: Solana smart contracts (called programs) are written in Rust.

3. Move (Aptos, Sui)

- New smart contract language created by Meta (for Diem project).
- Used in Aptos and Sui blockchains.
- Designed for safety in asset management.

4. Cairo (StarkNet)

- Language for StarkNet (a zk-Rollup on Ethereum).
- Optimized for zero-knowledge proofs.

5. Michelson (Tezos)

- A stack-based language for Tezos blockchain.
- Contracts are usually written in higher-level languages (like SmartPy in Python), then compiled down to Michelson.

6. Other Options

- Yul → low-level intermediate language for Ethereum (closer to assembly).
- Go, C#, Java, Kotlin → used in some non-EVM blockchains (e.g., Hyperledger, Neo).

Why Solidity is Still the Standard

- **EVM compatibility** → works on Ethereum, Polygon, Avalanche, BSC, Gnosis, Optimism, Arbitrum, etc.
- Huge ecosystem + tooling (Remix, Hardhat, Truffle).
- Most tutorials, libraries, and jobs are Solidity-based.

While Solidity is the most widely used language for EVM-compatible smart contracts, alternatives like Vyper (Ethereum, Python-based), Rust (Solana/Polkadot/NEAR), and Move (Aptos/Sui) also exist. Each language has trade-offs between security, performance, and ease of use. For my project, Solidity was chosen due to its maturity, ecosystem support, and compatibility across multiple test networks (Ethereum, Polygon, Avalanche, BSC, Gnosis).

Tools Used

- **Remix IDE** → for writing and compiling the contract.
- **MetaMask** → wallet to connect with test networks.
- **Testnet Faucets** → to get free ETH/MATIC/AVAX/BNB/xDAI for deployment.
- **Block Explorers** → to verify deployed contracts.

Deployment Steps

1. Compile in Remix

1. Open Remix IDE.
2. Create SimpleStorage.sol.
3. Write the contract.
4. Compile using **Solidity 0.8.x**.

2. Connect MetaMask

1. Install MetaMask browser extension.
2. Add the target testnet (Sepolia, Mumbai, Fuji, BSC, Chiado).
3. Fund wallet with test tokens from faucet.

3. Deploy

1. In Remix, go to **Deploy & Run Transactions**.
2. Environment → **Injected Provider – MetaMask**.
3. Select **SimpleStorage** contract.
4. Click **Deploy**.
5. Approve the MetaMask transaction.

4. Verify

1. Copy the **contract address** from Remix (Deployed Contracts section).
2. Paste into the testnet explorer (Etherscan, Polygonscan, Snowtrace, BscScan, Blockscout).
3. Confirm contract exists.

5. Interact

- Call getMessage() → returns "Hello".
- Call setMessage("My First DApp!") → confirm MetaMask.
- Call getMessage() again → returns "My First DApp!".

Network Configurations

Network	Chain ID	Currency	RPC URL	Faucet	Explorer
Ethereum Sepolia	11155111	ETH	https://sepolia.infura.io/v3/	Sepolia Faucet	Sepolia Etherscan
Polygon Mumbai/Amoy	80001 / 80002	MATIC	https://rpc-mumbai.maticvigil.com/ or https://rpc-amoy.polygon.technology/	Polygon Faucet	Mumbai Polygonscan / Amoy Polygonscan
Avalanche Fuji	43113	AVAX	https://api.avax-test.network/ext/bc/C/rpc	Avalanche Faucet	Snowtrace Fuji
BSC Testnet	97	BNB	https://data-seed-prebsc-1-s1.binance.org:8545/	BSC Faucet	BscScan Testnet
Gnosis Chiado	10200	xDAI	https://rpc.chiadochain.net	https://faucet.gnosischain.com/	Chiado Blockscout

Parameters Associated with Smart Contracts

When deploying or interacting with a smart contract, several key parameters are involved:

1. **Gas Fee**

- The cost of executing a transaction on the blockchain.
- Paid in the network's native token (ETH, MATIC, AVAX, BNB, xDAI).
- Depends on:
  - Gas Limit → maximum units of gas the transaction can consume.
  - Gas Price → cost per unit of gas (varies by network congestion).

2. **Transaction Hash (TxHash)**
   - A unique identifier for every transaction.
   - Can be used to track the transaction status on a block explorer.
3. **Block Number**
   - The block in which the transaction was confirmed.
   - Higher block numbers mean newer transactions.
4. **Contract Address**
   - A unique blockchain address generated when a smart contract is deployed.
   - This is where users and apps send transactions to interact with the contract.
5. **From / To Addresses;**
   - From → the wallet address initiating the transaction.
   - To → contract address (for contract calls) or another wallet (for transfers).
6. **Value (Optional)**
   - The amount of native cryptocurrency (e.g., ETH) sent with a transaction.
   - Our getter–setter contract does not use value transfers, but many contracts do.
7. **Nonce**
   - A sequential counter for each transaction from a wallet.
   - Ensures transactions are executed in the correct order.
8. **Status**
   - Indicates whether a transaction was successful or failed.